

Once you have the LXC toolset installed, you can check your default configuration with the following command:

```
$sudo lxc-checkconfig
```

Now, since we have LXC installed on our Ubuntu system, we can create our first container, as follows:

```
$sudo lxc-create -n mycontainer -t ubuntu
```

This will download an Ubuntu container from the Internet and save it in your storage system with the name *mycontainer*. You need to remember the *-n* option, as it will be used in most places to give your container a name. *-t template_name* defines the template we want to use. There are many other distribution templates available for LXC. You can find them at */usr/share/lxc/templates*.

You can see all the available containers you have created by using *lxc-ls*. Creating a container doesn't mean it is running. To run the container you need to use the *lxc-start* tool:

```
$sudo lxc-start -n mycontainer
```

This will start your first container in the background. You can check all the running containers by using *lxc-ls* also with the passing option *-fancy*.

This will show the state of all your containers with a few more specs. You can also use *lxc-top* to list the running container with the resources it uses.

Now, to start using your container, all you need to do is get attached to the container. To get direct access to the container as the root user without authentication, use the following command:

```
$sudo lxc-attach -n mycontainer
```

This is not always recommended due to the root login; so, in its place, you can use the following:

```
$sudo lxc-console -n mycontainer
```

This will give you the login prompt (default user name is *ubuntu* with password *ubuntu*). To detach from the container, just press *Ctrl-a*, followed by *q*.

If you are using LXC for development or testing purposes, then you have probably been in a situation in which you required an architecture system other than your current host. LXC has templates for most CPU architectures like i86, powerpc and armhf.

To create a container of a different architecture, you just need to add *--arch=* to the *lxc-create* command. Or you can even view the full list of templates and select one from them using the following command:

```
$sudo lxc-create --template download --name mycontainer
```

```
(Sat Jul-7 13:55:40)~(CPU 11.8%~Net 29)~(hackintosh:~)-(4.16.78)
$ ls /usr/share/lxc/templates
/          lxc-centos*      lxc-opensamlv*    lxc-sshd*
lxc-alpine* lxc-clr*         lxc-oracle*       lxc-ubuntu-cloud*
lxc-alpine* lxc-debian*      lxc-oracle*       lxc-ubuntu-cloud*
lxc-alllinux* lxc-download*   lxc-plan*
lxc-archlinux* lxc-fedora*     lxc-aladars*
lxc-busybox* lxc-gentoo*     lxc-sparcLinux*
```

Figure 1: Available templates for LXC

```
(Sat Jul-7 2:10:10am)~(CPU 11.5%~Net 29)~(hackintosh:~)-(4.16.78)
$ sudo lxc-ls -fancy
NAME      STATE   AUTOSTART GROUPS IPV4      IPV6
baseimage STOPPED 0       -         -
devstack  STOPPED 0       -         -
mycontainer RUNNING 0       -         10.0.3.151
ubuntu2   STOPPED 0       -         -
```

Figure 2: *lxc-ls -fancy* output, showing the state of containers with other details

Container Name	CPU	Mem	Swap	Used	Free	Used	Free	Used	Free	Used	Free
mycontainer	0.29	0.16	0.08	140.00 KB	12.86 MB	0.00	0.00	140.00 KB	12.86 MB	0.00	0.00
TOTAL 1 of 1	0.29	0.16	0.08	140.00 KB	12.86 MB	0.00	0.00	140.00 KB	12.86 MB	0.00	0.00

Figure 3: *lxc-top* output

```
(Sat Jul-7 12:10:10am)~(CPU 19.2%~Net 33)~(hackintosh:~)-(4.16.78)
$ sudo lxc-create --template download --name busybox1
Setting up the CPU keyring
Downloading the Image index
...
---
DIST      RELEASE ARCH      VARIANT BUILD
---
alpine    3.1    amd64    default    20170319_17:50
alpine    3.1    armhf    default    20161230_08:09
alpine    3.1    i386     default    20170319_17:50
alpine    3.2    amd64    default    20170504_18:43
alpine    3.2    armhf    default    20170319_17:50
alpine    3.2    i386     default    20170504_18:43
alpine    3.3    amd64    default    20170503_17:50
alpine    3.3    armhf    default    20170503_17:50
alpine    3.3    i386     default    20170503_17:50
alpine    3.4    amd64    default    20170504_18:09
alpine    3.4    armhf    default    20170511_20:27
alpine    3.4    i386     default    20170504_18:09
alpine    3.5    amd64    default    20170504_17:50
alpine    3.5    i386     default    20170503_17:50
alpine    edge   amd64    default    20170503_17:50
alpine    edge   armhf    default    20170511_20:27
alpine    edge   i386     default    20170504_18:09
archlinux current amd64    default    20170505_01:27
centos    6      amd64    default    20170504_02:16
centos    6      i386     default    20170504_02:16
centos    7      amd64    default    20170504_02:16
debian    Jessie amd64    default    20170504_02:41
debian    Jessie arm64    default    20170504_02:42
debian    Jessie armel    default    20170505_09:10
debian    Jessie armhf    default    20170504_02:41
debian    Jessie i386     default    20170505_04:24
debian    Jessie powerpc default    20170503_22:42
debian    Jessie ppc64el default    20170429_22:42
debian    Jessie s390x   default    20170504_22:42
debian    sid     amd64    default    20170504_22:42
debian    sid     arm64    default    20170504_22:42
debian    sid     armel    default    20170504_22:42
debian    sid     armhf    default    20170504_22:42
debian    sid     i386     default    20170504_22:42
debian    sid     powerpc default    20170504_22:42
debian    sid     ppc64el default    20170429_22:42
debian    sid     s390x   default    20170504_22:42
debian    stretch amd64    default    20170504_02:41
debian    stretch arm64    default    20170504_02:42
debian    stretch armel    default    20170505_04:24
debian    stretch armhf    default    20170504_02:41
debian    stretch i386     default    20170504_02:41
```

Figure 4: *lxc-create* can also be used to select templates on-the-fly

You can stop a container by using *lxc-stop*, freeze/pause a running container by using *lxc-freeze*, and then use *lxc-unfreeze* to unfreeze it. *lxc-destroy* will delete the container image. To use these tools, all you need to do is pass *-n containername* with these commands.

lxc-console or *lxc-attach* gives you access to the shell inside the container, but you may not always want that to happen. Often, you just want to run a single script or command inside the container. For that purpose, LXC provides *lxc-execute*.

```
$sudo lxc-execute -n mycontainer - ip add
```